



# Desenvolvimento Web

Profª Sabrina B. Moreira



# JavaScript

Aula 16.1: introdução, sintaxe, variáveis, tipos e alocação de memória

# JS: Introdução ao JavaScript

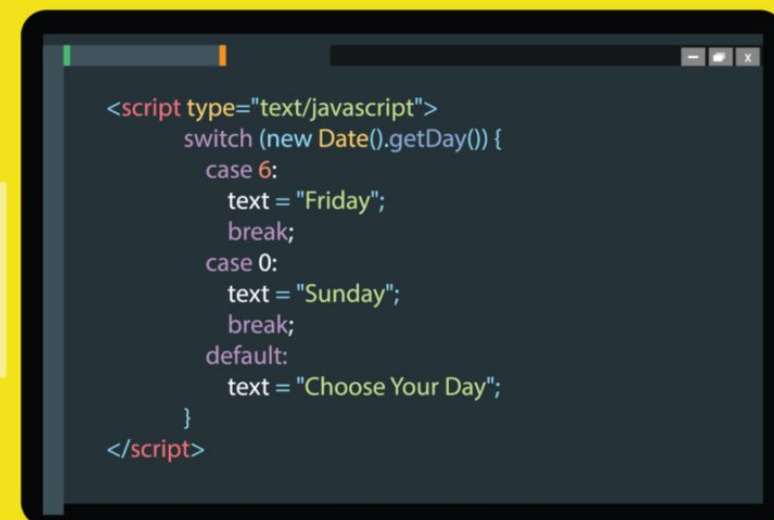
- ❖ JavaScript é uma linguagem de alto nível, dinâmica e interpretada.
- ❖ Pode ser usada para programação orientada a objetos ou funcional.
- ❖ Sua sintaxe é vagamente baseada na linguagem Java, mas, fora isso, não há outra relação entre as duas.



# JS: Introdução ao JavaScript

- ❖ Deixou para trás suas raízes como **linguagem de script** há muito tempo, tornando-se uma **linguagem de uso geral** robusta e eficiente, adequada para engenharia de software e grandes projetos.

## Javascript



# JS: Histórico

- ❖ Foi criado pela *Netscape* nos primeiros anos da Web.
- ❖ O nome "JavaScript" tornou-se uma **marca registrada** licenciada pela *Sun Microsystems* (posteriormente adquirida pela *Oracle*).
- ❖ Para garantir a padronização da **linguagem**, a *Netscape* submeteu sua especificação à *European Computer Manufacturers Association (ECMA)*.



Fonte: Silva, 2016

# JS: ECMAScript

- ❖ Como o nome JavaScript estava protegido por marca registrada, o padrão oficial recebeu o nome de "*ECMAScript*", embora a linguagem continue sendo popularmente conhecida como JavaScript.
- ❖ Desde a ES6, passou a adotar uma cadência de lançamento anual, e as versões da linguagem (ES2016, ES2017, ES2018, ES2019, ES2020...) hoje são identificadas pelo ano de lançamento.



Fonte: Noletto, 2021

# JS: Hospedeiro

- ❖ O ambiente **hospedeiro original** de execução do JavaScript é um navegador Web.
- ❖ O ambiente de navegador permite que o código JavaScript obtenha **entrada do usuário** por meio de mouse e teclado, além de fazer **requisições HTTP**.
- ❖ Ele também permite que o código JavaScript exiba saídas para o usuário.



# Sintaxe



# JS: Sintaxe



JS

```
x = 5
```

```
var x = 5;  
let x = 5;
```



JS

```
class Circle:
```

```
    def __init__(self, radius, color):  
        self.radius = radius  
        self.color = color
```

```
class Circle {  
    constructor(radius, color) {  
        this.radius = radius;  
        this.color = color;  
    }  
}
```

# JS: Sintaxe

## Nomenclaturas:

Segundo as diretrizes de JavaScript do MDN Web Docs:

- ❖ Em JS devemos usar o estilo de nomenclatura *lowerCamelCase*. O nome começa com uma **letra minúscula** e, em seguida, cada nova palavra começa com uma **letra maiúscula**. Diferente do Python que usa *snake\_case*.



`first_name`

`firstName`

# JS: Sintaxe

## Artigo complementar.

- ❖ Clique na imagem abaixo para visitar o artigo complementar sobre o tema.



Fonte: autora, 2026

# Conceitos iniciais

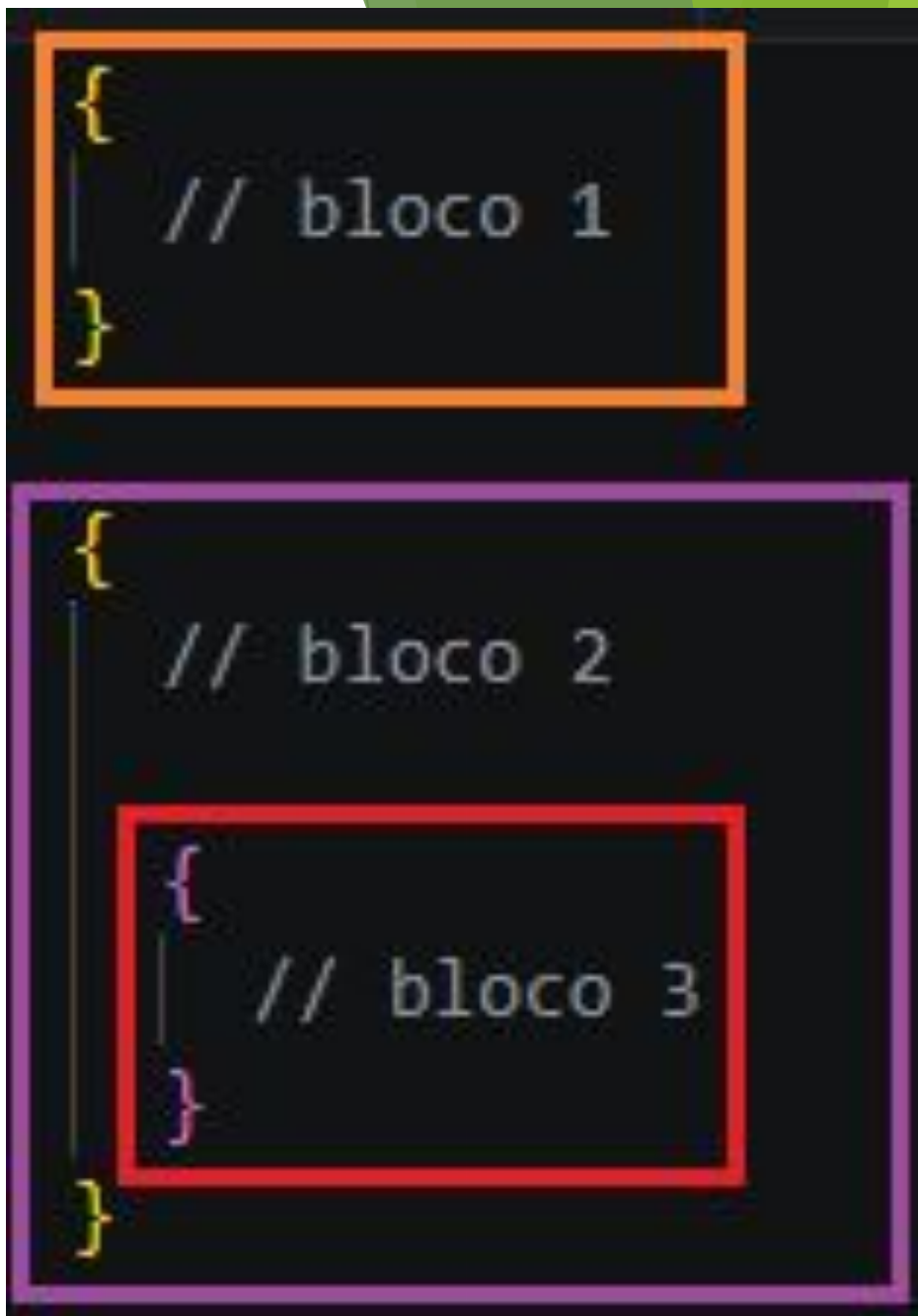
The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Bloco de código

# JS: blocos de código

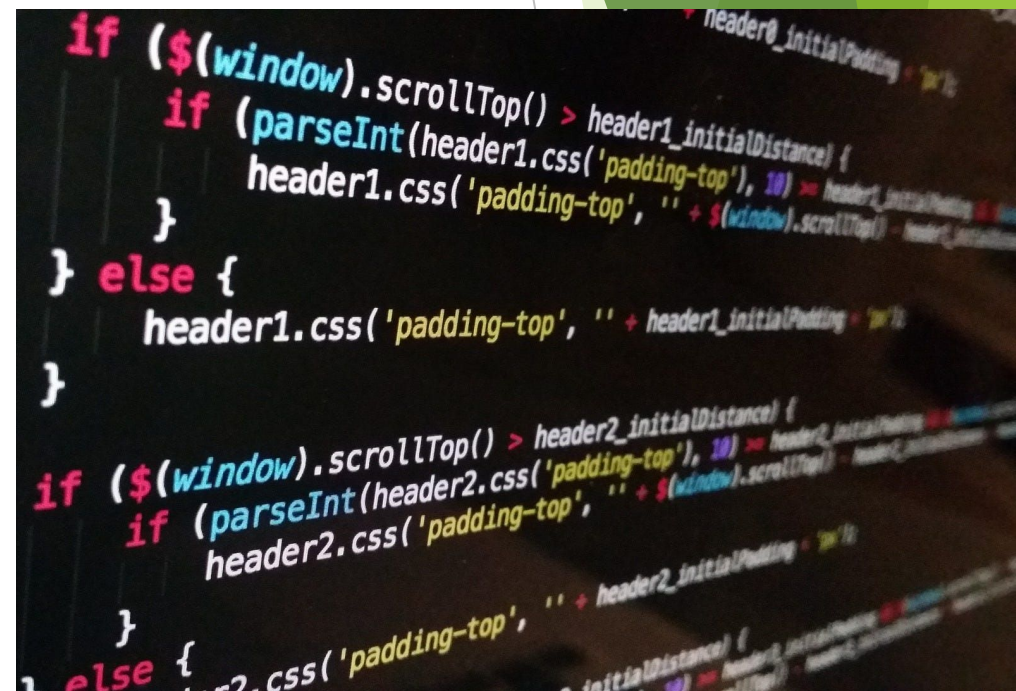
O que é um bloco de código?

- ❖ Um bloco de código é um conjunto de instruções agrupadas entre chaves { }.
- ❖ Essas chaves delimitam uma área do programa onde variáveis e comandos podem existir e ser executados.



# JS: blocos de código: por que é importante ?

- ❖ São uma das estruturas fundamentais do JavaScript e de muitas outras linguagens de programação.
- ❖ Eles são utilizados para agrupar instruções que pertencem ao mesmo contexto, tornando o código mais organizado, legível e fácil de manter.
- ❖ Além disso, controlam o escopo das variáveis, ou seja, **determinam** onde uma variável pode ser acessada.



```
if ($(window).scrollTop() > header1_initialDistance) {  
  if (parseInt(header1.css('padding-top'), 10) >= header1_initialPadding) {  
    header1.css('padding-top', '' + $(window).scrollTop() - header1_initialDistance);  
  } else {  
    header1.css('padding-top', '' + header1_initialPadding);  
  }  
  
if ($(window).scrollTop() > header2_initialDistance) {  
  if (parseInt(header2.css('padding-top'), 10) >= header2_initialPadding) {  
    header2.css('padding-top', '' + $(window).scrollTop() - header2_initialDistance);  
  } else {  
    header2.css('padding-top', '' + header2_initialPadding);  
  }  
  
if ($(window).scrollTop() > header3_initialDistance) {  
  if (parseInt(header3.css('padding-top'), 10) >= header3_initialPadding) {  
    header3.css('padding-top', '' + $(window).scrollTop() - header3_initialDistance);  
  } else {  
    header3.css('padding-top', '' + header3_initialPadding);  
  }  
}
```

Fonte: Nascimento, 2021

# Variáveis



# JS: variáveis

- ❖ Em programação, uma variável é um **espaço na memória** utilizado para armazenar informações que poderão ser usadas durante a execução do programa.



# Alocação de memória

# JS: variável e alocação de memória

- ❖ Quando uma variável é criada, o mecanismo da linguagem reserva um **espaço na memória ram** para armazenar o valor associado a ela.
- ❖ Esse processo é chamado de **alocação de memória**.
- ❖ Dependendo do tipo de dado, o valor pode ser armazenado diretamente ou por meio de uma referência para outra região da memória.

Variable Name



num

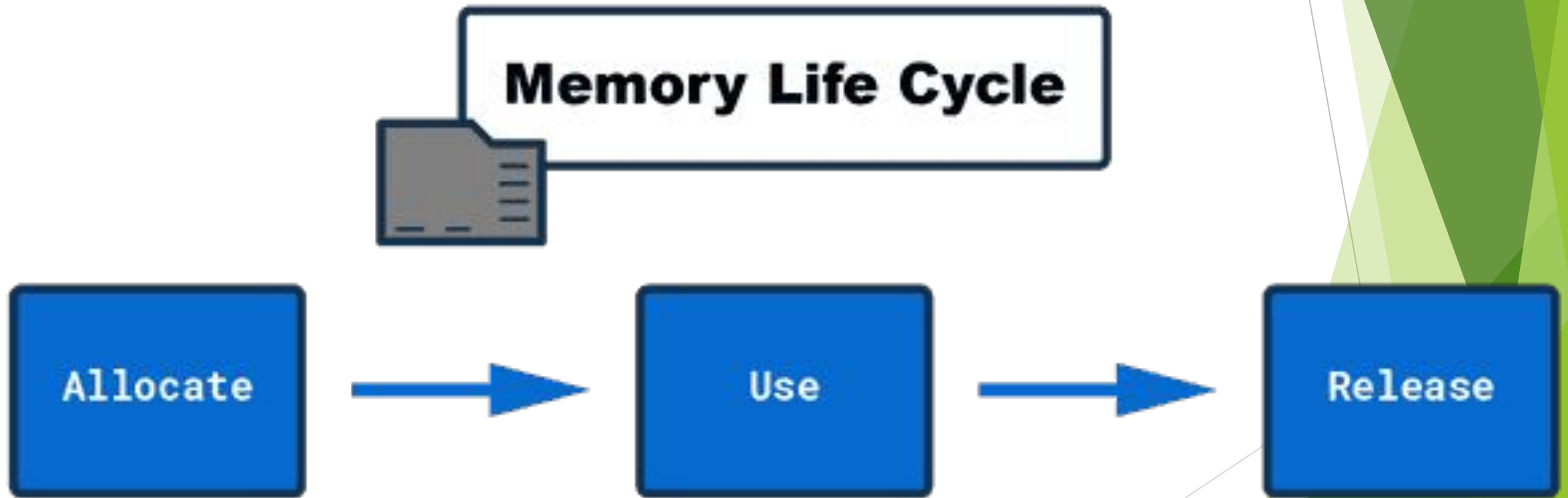


Value in  
memory  
location 24276

24276

Memory  
address

# JS: ciclo de vida da memória



# JS: ciclo de vida da memória

## ❖ Alocação de memória (Allocate memory)

Quando criamos uma variável, objeto ou outra estrutura de dados, o mecanismo da linguagem **aloca** (reserva) a quantidade de memória necessária para armazenar essas informações.

## ❖ Uso da memória (Use memory)

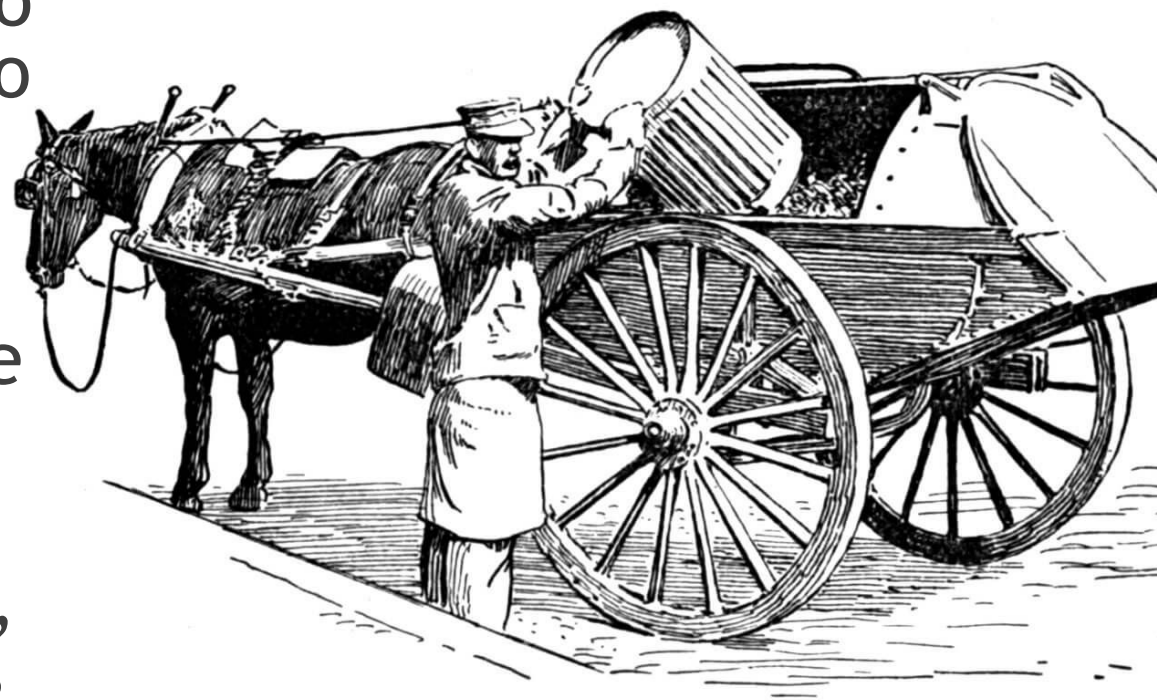
Ao ler ou alterar o valor de uma variável, realizando operações de **leitura e escrita** em áreas da memória onde esses dados estão armazenados.

## ❖ Liberação de memória (Release memory)

Quando uma área de memória não é mais **necessária** e não possui **referências ativas**, ela pode ser liberada pelo coletor de lixo (Garbage Collector), liberando esse espaço para armazenar novos dados.

# JS: ciclo de vida da memória

- ❖ O desenvolvedor *JavaScript* não precisa se preocupar em **reservar** ou **liberar memória** manualmente. Isso é gerenciado automaticamente pelo **Garbage Collector**, que remove da memória os dados que não estão mais sendo utilizados, similar a linguagens como *Java*, *C#*, *Python* e *Ruby*.
- ❖ Já o *C* e *C++* necessita que o desenvolvedor gerencie a memória, o que oferece **maior controle**, mas aumenta a complexidade e o **risco de erros**.





# JS: declaração de variáveis

No JavaScript, existem três formas de declarar variáveis:

- ◆ *var*

- ◆ *let*

- ◆ *const*

Cada uma possui características específicas e deve ser utilizada em situações diferentes.



The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern, layered effect. The word "Var" is centered in a green, sans-serif font.

Var



## JS: var

- ❖ A palavra-chave **var** era a única forma de declarar variáveis nas versões mais antigas do JavaScript. Atualmente seu uso não é recomendado mas é importante conhecê-la para entender códigos legados.
- ❖ Uma variável declarada com **var** pode ter seu valor alterado durante a execução do programa (**reatribuição**).

```
{  
  // Bloco de código 1  
  var myName; // declaração da variável bloco 1  
  myName = "Ana"; // atribuição de valor no bloco 1  
  console.log("Meu nome é: " + myName); // acesso do valor da variável no bloco 1  
}
```

Meu nome é: Ana

## JS: var e bloco de código

- ❖ Uma variável declarada em um bloco de código, pode ser vista em outro, vazando seu valor entre os blocos. No exemplo abaixo, **não temos** erros algum ao tentar acessar no **bloco 2** o valor da variável que foi declarada (e só deveria existir) no **bloco 1**.

```
> {  
  {  
    // Bloco de código 1  
    var myName = "Ana"; // declaração da variável bloco 1  
  }  
  // Bloco de código 2  
  console.log("Meu nome é: " + myName); // var vaza o valor da variável entre os blocos  
}
```

Meu nome é: Ana

## JS: var

- ❖ Outra característica do var é que ele permite que uma variável seja **declarada novamente** com o mesmo nome (**redeclaração**) no mesmo bloco de código. O que pode gerar erros difíceis de identificar.

```
{
  {
    // Bloco de código 1
    var myName; // declaração da variável bloco 1
    console.log("Meu nome é: " + myName); // acesso de variável no bloco 1
    var myName = "Ana"; // redeclaração de variável com mesmo nome no bloco 1
    console.log("Meu nome é: " + myName); // acesso de variável no bloco 1
  }
  // Bloco de código 2
  console.log("Meu nome é: " + myName); // acesso de variável no bloco 2
  var myName = "Maria"; // redeclaração de variável com mesmo nome no bloco 2
  console.log("Meu nome é: " + myName); // acesso de variável no bloco 2
}
```

Meu nome é: undefined

Meu nome é: Ana

Meu nome é: Ana

Meu nome é: Maria

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern, layered effect. The shapes are concentrated on the left and right sides, leaving a large white central area.

Let

# JS: let

- ❖ A palavra-chave *let* foi introduzida em 2015 com o ECMAScript 6 (ES6) para substituir grande parte do uso do *var*.
- ❖ Assim como o *var*, uma variável declarada com *let* pode ter seu valor alterado (**reatribuição**).

```
{
  // Bloco de código 1
  let myName; // declaração da variável bloco 1
  myName = "Ana"; // atribuição de valor no bloco 1
  console.log("Meu nome é: " + myName); // acesso do valor da variável no bloco 1
}
```

Meu nome é: Ana



## JS: let e bloco de código

- ❖ Diferentemente do `var`, com o `let`, uma **variável declarada** em um bloco de código, **NÃO** pode ser acessada em **outro bloco**, e caso seja tentando realizar esse acesso, ocorrerá erro do exemplo abaixo: *“Uncaught ReferenceError: myName is not defined”*.

```
> {  
  {  
    // Bloco de código 1  
    let myName = "Ana"; // declaração da variável bloco 1  
  }  
  // Bloco de código 2  
  console.log("Meu nome é: " + myName); // tentativa de acesso da variável no bloco 2  
}
```

✖ ▶ Uncaught ReferenceError: myName is not defined

## JS: let e bloco de código

- ❖ Outra diferença do *var* e do *let*, é que o *let* **NÃO** permite redeclaração de **variável de mesmo nome** no mesmo bloco de código, causando o erro “*Uncaught SyntaxError: Identifier 'myName' has already been declared*”.

```
{
  {
    // Bloco de código 1
    let myName; // declaração da variável bloco 1
    let myName = "Ana"; // redeclaração de variável com mesmo nome no bloco 1
  }
  // Bloco de código 2
}
```

❌ Uncaught SyntaxError: Identifier 'myName' has already been declared

# JS: let e bloco de código

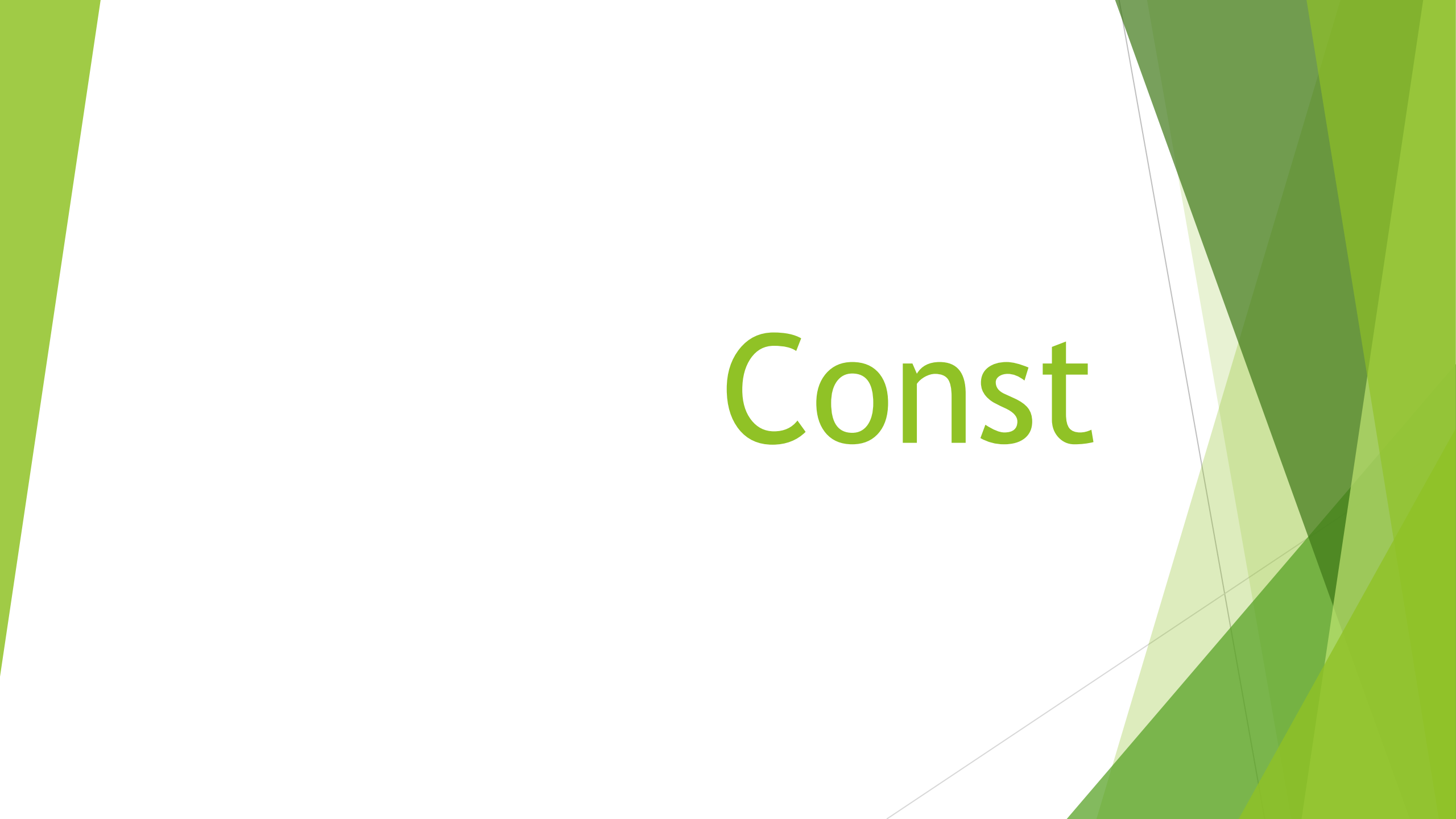
- ❖ Já para blocos de códigos diferentes é possível realizar a redeclaração de variáveis de mesmo nome.

```
{
  {
    // Bloco de código 1
    let myName; // declaração da variável bloco 1
    console.log("Meu nome é: " + myName);
  }
  // Bloco de código 2
  let myName = "Ana"; // redeclaração de variável com mesmo nome no bloco 2
  console.log("Meu nome é: " + myName);
}
```

Meu nome é: undefined

Meu nome é: Ana



The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

# Const

## JS: const

- ❖ A palavra-chave **const** é utilizada para criar constantes.
- ❖ Ou seja, valores que não devem ser alterados após sua criação.



## JS: const

- ❖ Ao declarar uma constante, é obrigatório atribuir um valor **IMEDIATAMENTE**.
- ❖ Caso não seja atribuído um valor ao declarar a variável constante, ocorrerá o erro “*Uncaught SyntaxError: Missing initializer in const declaration*”.

```
> {  
  // Bloco de código 1  
  const myName; // declaração e sem atribuição de valor no bloco 1  
}
```

✖ Uncaught SyntaxError: Missing initializer in const declaration

## JS: const

- ❖ Do mesmo jeito que o *let*, o *const* **NÃO** aceita redeclarar uma variável de mesmo nome no mesmo bloco de código, somente em blocos diferentes.

```
{
  {
    // Bloco de código 1
    const myName = "Ana"; // declaração da variável bloco 1
    console.log("Meu nome é: " + myName);
  }
  // Bloco de código 2
  const myName = "Maria"; // redeclaração de variável com mesmo nome no bloco 2
  console.log("Meu nome é: " + myName);
}
```

## JS: Quando se deve usar *const*?

- ❖ Quando um valor NÃO mudar durante a execução do programa, o uso de *const* é adequado.
- ❖ Caso seja realizado a reatribuição de um novo valor a uma constante, ocorrerá um o erro “*Uncaught TypeError: Assignment to constant variable.*”;

Para essa situação, deve ser usado o *let*. Para todas as outras, use o *const*. E *var*, NÃO utilize.

```
{  
  // Bloco de código 1  
  const myName = "Ana"; // declaração e atribuição de valor no bloco 1  
  myName = "maria"; // reatribuição de valor no bloco 1  
}
```

Uncaught TypeError: Assignment to constant variable.

## JS: var x let x const

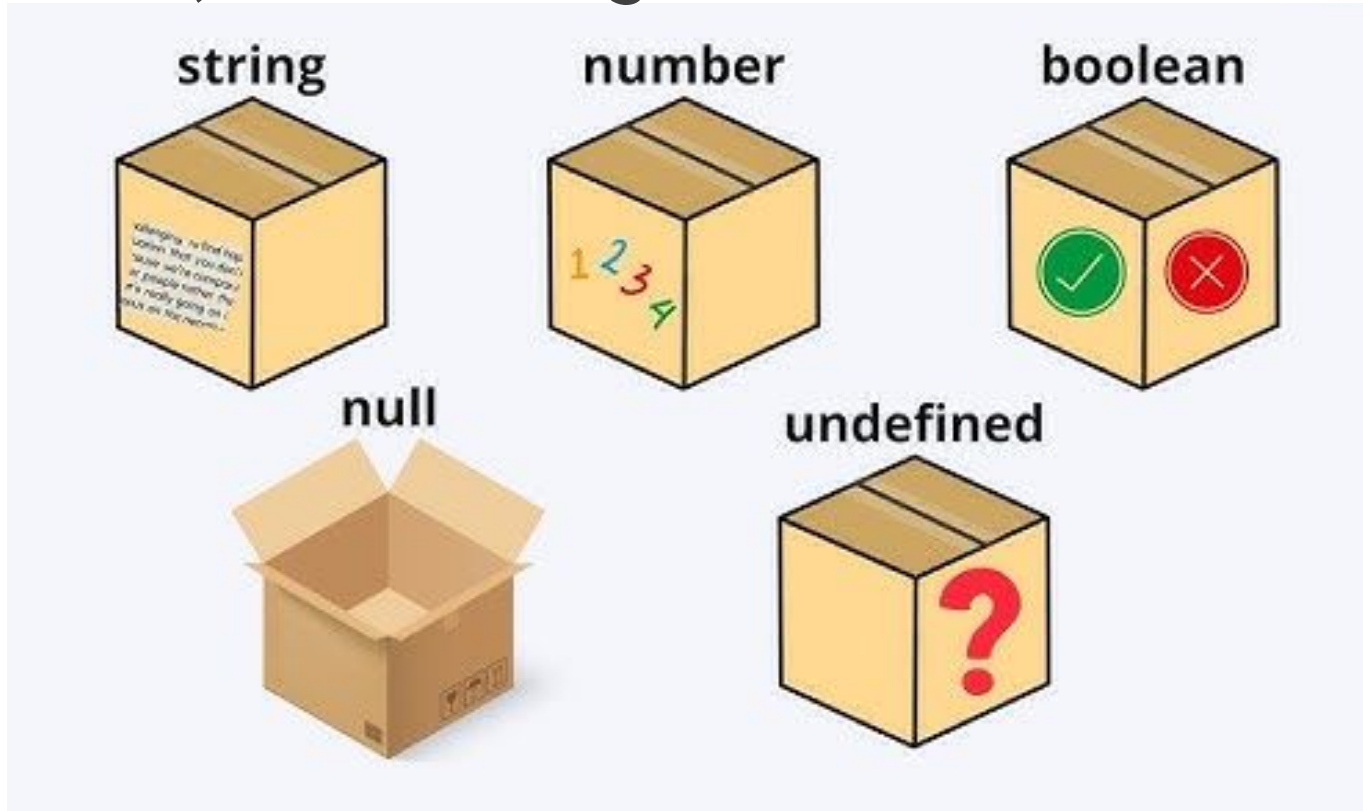
| Característica                   | var | let | const |
|----------------------------------|-----|-----|-------|
| Pode alterar o valor?            | Sim | Sim | Não   |
| Pode redeclarar?                 | Sim | Não | Não   |
| Uso recomendado atualmente?      | Não | Sim | Sim   |
| Precisa atribuir valor ao criar? | Não | Não | Sim   |

# Tipos de variáveis

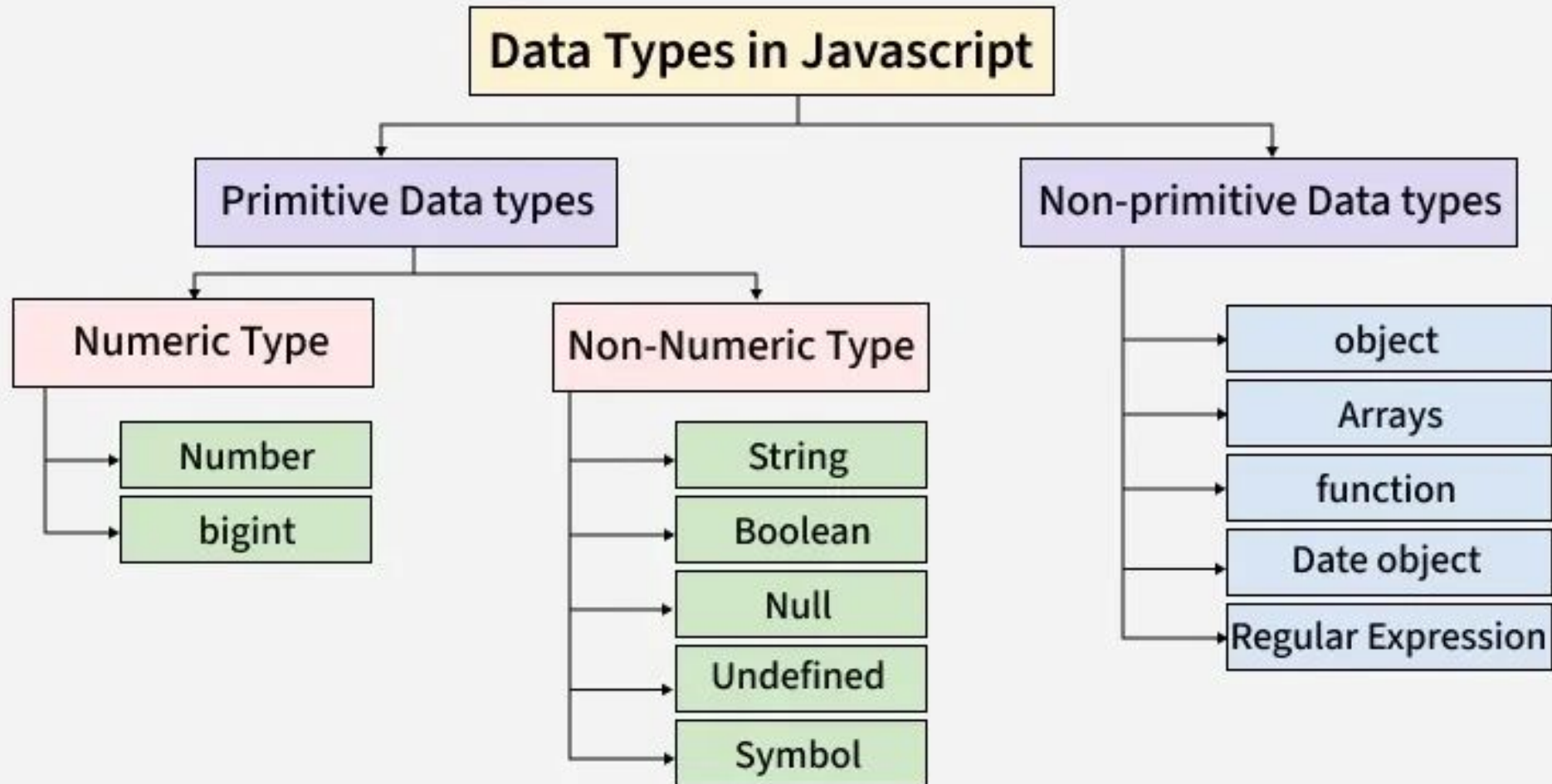


# JS: tipos de variáveis

- ❖ As variáveis podem armazenar diferentes tipos de **dados**, permitindo representar informações como números, textos, valores lógicos e estruturas mais complexas.



# JS: todos os tipos de variáveis



# JS: os principais tipos de variáveis

Os tipos mais comuns são:

- ❖ string (textos)
- ❖ number (valores numéricos inteiros ou decimais)
- ❖ boolean (true ou false)
- ❖ object (agrupamento de dados relacionados)
- ❖ null (ausência intencional de valor)
- ❖ undefined (variável declarada ainda sem valor atribuído)

# JS: os principais tipos de variáveis

|         |               |           |              |       |          |     |
|---------|---------------|-----------|--------------|-------|----------|-----|
| String  | "hello world" | 'abacaxi' | `5 patinhos` |       |          |     |
| Number  | 20            | 3.1415    | -18          | -9.04 | Infinity | NaN |
| Boolean | true          | false     |              |       |          |     |
| Object  | {}            |           |              |       |          |     |
| Outros  | null          | undefined |              |       |          |     |

# Tipos de variáveis especiais

# JS: tipos especiais de variáveis

- ❖ Quando falamos em tipos especiais geralmente estamos nos referindo aos valores que não se encaixam nos tipos tradicionais como *string*, *number* ou *boolean*.
- ❖ Mas que possuem comportamentos próprios e merecem atenção especial.

NaN  NaN

# JS: tipos especiais de variáveis

| Nome      | Tipo      | Descrição                                                 | Exemplo                                                |
|-----------|-----------|-----------------------------------------------------------|--------------------------------------------------------|
| undefined | undefined | Variável declarada, mas sem valor atribuído               | let name; ou let name = undefined;                     |
| null      | object    | Representa ausência intencional de valor                  | let name = null;                                       |
| NaN       | number    | Resultado de uma operação matemática inválida             | "abc" * 2; ou (0 / 0 );                                |
| Infinity  | number    | Representa infinito positivo                              | 1 / 0                                                  |
| -Infinity | number    | Representa infinito negativo                              | -1 / 0                                                 |
| BigInt    | bigint    | Números inteiros maiores que o suportado pelo tipo number | 100n                                                   |
| Symbol    | symbol    | Identificador único e imutável                            | const id1 = Symbol("id");<br>const id2 = Symbol("id"); |



# JS: tipo especial NaN

- ❖ O NaN (acrônimo para **Not-a-Number**, ou "não é um número") é um valor numérico especial que indica um **resultado inválido** ou **indefinido** após uma operação matemática falhar.
- ❖ Ex: tentar dividir zero por zero ou converter uma string de texto em número).

```
> x = "a" - "b"; // NaN
< NaN

> x = 0 / 0; // NaN
< NaN

> x = parseInt("oi"); // NaN
< NaN

> x = 1 + NaN; // NaN
< NaN
```

## JS: tipo especial NaN

- ❖ Além do *NaN* ser do tipo numérico.
- ❖ Uma outra característica estranha do *NaN* é que ele é o único valor que não é igual a ele mesmo ao fazer comparações de nenhum tipo.
- ❖ Isso acontece porque o *NaN* representa um “resultado indefinido ou inválido”, então o JavaScript não consegue considerar dois *NaN* como valores equivalentes.

```
> NaN == NaN;  
< false  
> NaN === NaN;  
< false  
> NaN != NaN;  
< true  
> NaN !== NaN;  
< true
```

## JS: função global isNaN()

- ❖ A função `isNaN()` foi criada para **identificar** quando o resultado de uma operação matemática **não era um número válido**, ou seja, quando o valor retornado era NaN. Significa literalmente “*isso é Not a Number?*”
- ❖ O uso dessa função deve acontecer **exclusivamente** para valores numéricos ou que podem ser **convertidos corretamente** aos mesmos.

```
let stringToNumber = Number("oi");  
NaN  
isNaN(Number("oi"));  
true  
isNaN(stringToNumber);  
true  
isNaN(NaN);  
true  
isNaN("0/0");  
true  
isNaN("25/0");  
true  
isNaN("5");  
false
```

## JS: função `Number.isNaN()`

- ❖ Pela busca por comportamentos mais previsíveis, o *ECMAScript 2015 (ES6)* introduziu a função *Number.isNaN()*.
- ❖ Seu objetivo é verificar de forma mais **segura e rigorosa** se um valor é realmente o valor especial *NaN*, sem realizar **conversões automáticas**.
- ❖ Para verificar corretamente deve ser usada a função global *Number.isNaN()*, passando entre os parênteses o valor a ser verificado.

```
> Number.isNaN(0 / 0);  
< true  
> Number.isNaN(Number("a"));  
< true  
> Number.isNaN(1);  
< false  
> Number.isNaN("oi");  
< false  
> Number.isNaN(true);  
< false  
> Number.isNaN("a" + "b");  
< false  
> Number.isNaN("a" + 3);  
< false  
> Number.isNaN(3 + "1");  
< false  
> Number.isNaN(3 - "1");  
< false
```

# JS: isNaN() vs Number.isNaN()

| Função         | Faz conversões automáticas? | Seu uso é recomendado?                                                                             | Exemplo                                                                 |
|----------------|-----------------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| isNaN()        | Sim                         | Raramente. Somente quando temos certeza que um valor pode ser convertido a um número corretamente. | isNaN("abc");<br>// true, falso positivo<br>isNaN("0/0"); true, bom uso |
| Number.isNaN() | Não                         | Sim, em todas as situações.                                                                        |                                                                         |

Fonte: autora, 2026



## JS: isNaN() vs Number.isNaN()

- ❖ Podemos optar pelo *isNaN()* para os casos abaixo no qual a **conversão automática economiza** trabalho, fora isso, prefira utilizar o *Number.isNaN()*.

```
// uso indicado do isNaN()
isNaN("NaN"); // converte automaticamente a string NaN em número
true

Number.isNaN("NaN"); // não faz conversão automática e independente do valor, o tipo string não pode ser um NaN
false

Number.isNaN(Number("NaN")); //Aqui precisaríamos fazer a conversão manual para ter o mesmo resultado do isNaN
true
```

# JS: tipo especial Symbol

- ❖ É utilizado para **criar identificadores únicos**. Mesmo variáveis tenham a **mesma descrição**, **nunca serão consideradas iguais**.
- ❖ Isso acontece porque cada chamada de um *Symbol()* é associado a ele um valor exclusivo.
- ❖ É utilizado para criação de propriedades privadas ou semiprivadas em objetos, desenvolvimento de bibliotecas e frameworks e evitar conflitos de nomes entre diferentes partes da aplicação.

```
const id1 = Symbol(1);  
const id2 = Symbol(1);  
console.log(id1 == id2);  
  
false  
  
console.log(id1 === id2);  
  
false  
  
const id3 = Symbol.for(1);  
const id4 = Symbol.for(1);  
console.log(id3 === id4);  
  
true
```



## JS: tipo especial *Symbol*

- ❖ O tipo *Symbol*, permite criar identificadores únicos em JavaScript, sendo útil para evitar conflitos de nomes em objetos e bibliotecas.
- ❖ Cada chamada de *Symbol()* gera um **valor exclusivo**, mesmo que vários destes utilizem a mesma descrição.
- ❖ Já *Symbol.for()* utiliza um **registro global** de símbolos: se um símbolo com a descrição informada já existir, ele será **reutilizado**, caso contrário, será criado um novo e armazenado.
- ❖ Assim, *Symbol()* é usado para **criar identificadores sempre únicos**, enquanto *Symbol.for()* é indicado quando diferentes partes da aplicação precisam **compartilhar o mesmo símbolo**.



# JavaScript

Aula 16.2: tipagem dinâmica, tipos de variáveis, expressões e operadores

# Tipagem dinâmica

# JS: tipagem dinâmica

- ❖ O JavaScript possui tipagem dinâmica, o que significa que uma **mesma variável pode armazenar diferentes tipos** de dados ao longo da execução do programa.

```
x = 1; // Número
x = 0.01; // Número inteiro ou real.
x = "hello world"; // Strings de texto entre aspas duplas.
x = 'JavaScript'; // Aspas simples também delimitam strings.
x = true; // Um valor booleano.
x = false; // O outro valor booleano.
x = null; // Significa "não há valor aqui".
x = undefined; // Significa "nenhum valor foi definido ainda".
```

# Expressão e operadores

# JS: expressão e operadores

- ❖ Uma **expressão** é uma frase em JavaScript que pode ser avaliada para **produzir um valor**.
- ❖ Uma das maneiras mais comuns de **formar expressões** em JavaScript é usar operadores.
- ❖ Os tipos de operadores são: *aritméticos, atribuição, comparação, lógicos e incremento / decremento*.



**OPERADORES**



Fonte: Pereira, 2023

# Operadores aritméticos



# JS: operadores aritméticos

- ❖ São utilizados para realizar **cálculos matemáticos**.
- ❖ Como adição, subtração, multiplicação, divisão, resto e potência.

```
3 + 2; // adição, resultado é 5  
3 - 2; // subtração, resultado é 1  
3 * 2; // multiplicação, resultado é 6  
3 / 2; // divisão, resultado é 1.5  
3 % 2; // módulo, resultado é 1 (resto da divisão de 3 por 2)  
3 ** 2; // exponenciação, resultado é 9 (3 elevado a 2)
```

# JS: operadores aritméticos

| Operador | Descrição                    | Exemplo | Resultado |
|----------|------------------------------|---------|-----------|
| +        | Soma                         | 10 + 5  | 15        |
| -        | Subtração                    | 10 - 5  | 5         |
| *        | Multiplicação                | 10 * 5  | 50        |
| /        | Divisão                      | 10 / 5  | 2         |
| %        | Resto da divisão<br>(módulo) | 10 % 3  | 1         |
| **       | Potência                     | 2 ** 3  | 8         |

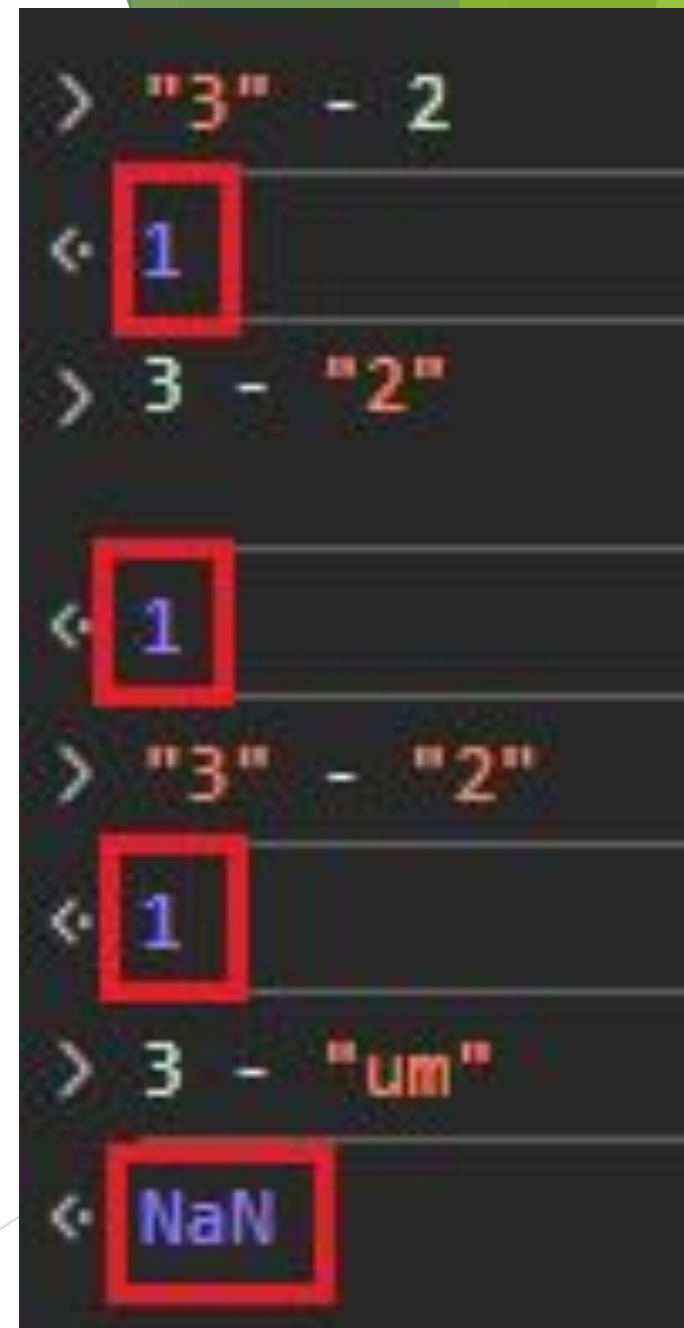
## JS: operadores aritméticos: soma

- ❖ Podemos utilizar a soma para fazer a adição de valores **numéricos** ou para concatenar **textos** também.
- ❖ Concatenar em JavaScript significa unir duas ou mais sequências de caracteres (*strings*) ou *arrays* para criar um **novo elemento combinado**.

```
> 3+2
< 5
> "3"+2
< '32'
> "3"+"2"
< '32'
> 3-2
< 1
> "3"-2
< 1
> "3"- "2"
< 1
> "a"+"b"+"c"
'abC'
```

## JS: operadores aritméticos

- ❖ Todos os outros operadores (exceto +) só funcionam em operações aritméticas.
- ❖ Caso seja utilizado para **textos**, o JavaScript tentará converter automaticamente os valores **não numéricos** para números antes de executar o cálculo.
- ❖ Essa conversão automática é chamada de **coerção de tipos** (*type coercion*).



```
> "3" - 2
< 1
> 3 - "2"
< 1
> "3" - "2"
< 1
> 3 - "um"
< NaN
```

The screenshot shows a JavaScript console with three pairs of input and output. In each pair, the input is on the line starting with '>' and the output is on the line starting with '<'. The outputs 1, 1, and 1 are highlighted with red boxes. The output NaN is also highlighted with a red box.

# Operadores de atribuição

# JS: operadores de atribuição

- ❖ Servem para armazenar ou atualizar valores em variáveis.
- ❖ Podem ser usados de forma combinada com operadores aritméticos.

```
let count = 1; // atribuição de valor
count += 2; // soma mais 2 em count, count agora é 3
count -= 1; // subtrai 1 do valor de count, agora é 2
count *= 2; // multiplica count por 2, agora é 4
count /= 2; // divide valor de count por 2, count agora é 2
count %= 3; // atribui o resto da divisão de count por 3, count agora é 2
count **= 3; // eleva count a potência de 3, count agora é 8
```

# JS: operadores de atribuição

| Operador | Descrição            | Exemplo     |
|----------|----------------------|-------------|
| =        | Atribui um valor     | numero = 20 |
| +=       | Soma e atribui       | numero += 5 |
| -=       | Subtrai e atribui    | numero -= 3 |
| *=       | Multiplica e atribui | numero *= 2 |
| /=       | Divide e atribui     | numero /= 2 |

Fonte: autora, 2026



# Operadores de comparação

# JS: operadores de comparação

Comparam valores e retornam:

- ❖ true
- ❖ false

Indicando que o resultado da comparação foi verdadeira ou falsa.



# JS: operadores de comparação

| Operador | Descrição                 | Exemplo   | Resultado |
|----------|---------------------------|-----------|-----------|
| ==       | Compara apenas valores    | 5 == "5"  | true      |
| ===      | Compara valor e tipo      | 5 === "5" | false     |
| !=       | Difere os valores         | 5 != "5"  | false     |
| !==      | Difere os valores e tipos | 5 !== "5" | true      |
| >        | Maior que                 | 10 > 5    | true      |
| <        | Menor que                 | 5 < 10    | true      |
| >=       | Maior ou igual            | 10 >= 10  | true      |
| <=       | Menor ou igual            | 5 <= 10   | true      |

# Comparação de igualdade

# JS: comparação simples

- ❖ Utiliza os operadores `==` e `!=`. Esses operadores fazem uma **conversão automática de tipos antes de comparar os valores**, utilizando o algoritmo *Abstract Equality Comparison*.

```
// Comparação simples
5 == "5"

// O algoritmo identifica comparação de tipos
// diferentes e converte a string "5" para um número 5
5 == Number("5");

// e segue a comparação somente dos valores.
5 == 5; // true
```

# JS: comparação estrita

- ❖ A palavra estrito significa que a comparação é mais rigorosa.
- ❖ Utiliza os operadores `==` e `!=`.

O JavaScript exige que:

- ❖ os valores sejam iguais.
- ❖ os tipos também sejam iguais.

Caso contrário, o resultado será `false`.

```
// Comparação simples
// converte a string para número e compara
5 == "5" // true

// Comparação estrita
// não há conversão de tipos, compara os
// tipos e valores existentes.
5 === "5" // false
```

Fonte: autora, 2026

# JS: Comparações no JavaScript

Artigo complementar

❖ Clique na imagem abaixo para ler.



Fonte: autora, 2026



# Comparação de diferenças

# JS: diferença simples

Assim como existem duas formas de verificar igualdade também existem duas formas de verificar se valores são diferentes:

- ❖ `!=` diferença simples
- ❖ `!==` diferença estrita

```
// Diferença simples
5 != "5"; // false, ambos os valores são iguais (conversão de tipo para validar valor)

// Diferença estrita
5 !== "5"; // true, seus valores ou tipos são diferentes
```

# JS: operadores de comparação

Prefira usar as comparações estritas:

- ❖ === (igual estrito)
- ❖ !== (diferente estrito)

Esses operadores evitam conversões automáticas de tipos e tornam o código mais previsível e seguro.

# Outros operadores de comparação

# JS: outros operadores de comparação

- ❖ Maior que (>): verifica se o valor à esquerda é maior que o valor à direita.
- ❖ Menor que (<): verifica se o valor à esquerda é menor que o valor à direita.
- ❖ Maior ou Igual (>=): verifica se o valor à esquerda é maior ou igual ao valor à direita.
- ❖ Menor ou Igual (<=): Verifica se o valor à esquerda é menor ou igual ao valor à direita.

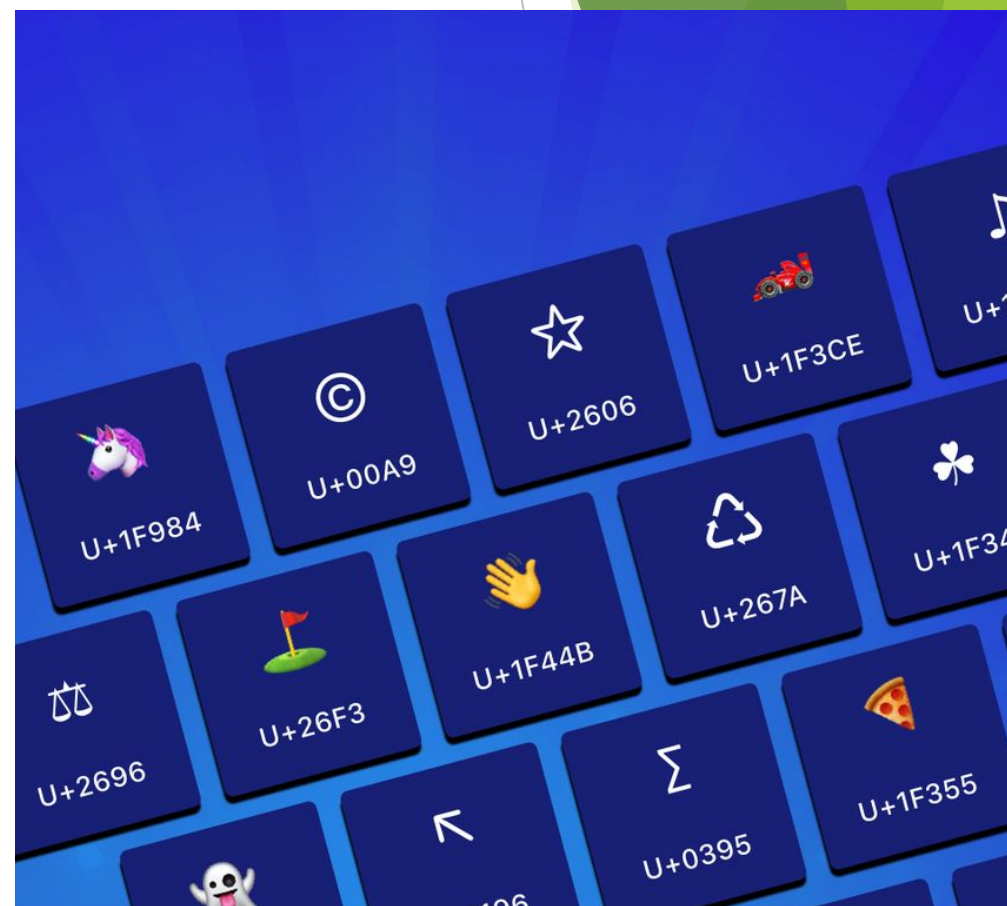
```
5 > 1; // true, 5 é maior que 1
5 < 1; // false. 5 não é menor que 1
5 >= 5; // true, 5 é maior ou igual a 5
4 <= 5; // true, 4 é menor ou igual a 5
```

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

# Unicode

# JS: Unicode

- ❖ Quando duas **strings** são **comparadas**, é analisado seus caracteres da esquerda para a direita usando valores *Unicode*.
- ❖ O *Unicode* é um padrão que atribui um número único em hexadecimal para **cada caractere** utilizado em computadores, para os representar em diversos idiomas.
- ❖ O valor Unicode não tem relação com o tamanho de memória usado, pois isso depende da codificação (UTF-16).



Fonte: Tasker, 2021



# JS: Unicode

| Caracter                                                                            | Unicode | Hexadecimal | Decimal | Binário               |
|-------------------------------------------------------------------------------------|---------|-------------|---------|-----------------------|
| a                                                                                   | U+0061  | 0x61        | 97      | 01100001              |
| b                                                                                   | U+0062  | 0x62        | 98      | 01100010              |
| c                                                                                   | U+0063  | 0x63        | 99      | 01100011              |
| A                                                                                   | U+0041  | 0x41        | 65      | 01000001              |
|   | U+2696  | 0x2696      | 9878    | 10011010010110        |
|  | U+1F600 | 0x1F600     | 128512  | 11111011000000<br>000 |

# Comparação de strings

# JS: comparação de strings puras

- ❖ Comparação de *strings* puras é quando todos os valores comparados são *strings*.
- ❖ Nesse cenário, tanto `==` quanto `===` acabam produzindo o **mesmo resultado** porque não existe necessidade de **conversão de tipo** (coerção).
- ❖ É apenas comparado os caracteres das strings usando o valor Unicode, verificando se são exatamente iguais na **sequência**.

```
console.log("a" == "a");  
true  
console.log("a" === "a");  
true  
console.log("a" != "a");  
false  
console.log("a" !== "a");  
false
```

# JS: comparação de strings puras

- ❖ O JavaScript compara os **caracteres das strings** da esquerda para a direita usando seus valores Unicode (decimal).
- ❖ A comparação **termina** na primeira diferença encontrada.

```
"A" < "a" // true, A (65) é menor que a (97)
"casa" > "carro" // true, casa é maior que carro
// Comparação:
// c = c (99)
// a = a (97)
// s (115) > r (114)
// diferença encontrada, comparação finalizada
```

Fonte: autora, 2026

# JS: comparação de strings puras

- ❖ Se todos os caracteres comparados forem iguais, a string mais longa será considerada maior. Maiúsculas e minúsculas são diferenciadas.

```
"java" < "javascript" // true, java é menor que javascript
// Comparação:
// j = j (106)
// a = a (97)
// v = v (118)
// a = a (97)
// A string java termina antes de javascript, portando é
// considerada menor.
```

# JS: Unicode

Artigo complementar

- ❖ Clique na imagem ao lado para ler.



# Operadores de lógicos



# JS: operadores lógicos

- ❖ São utilizados para **combinar condições**, retornando o resultado da comparação como verdadeiro (true) ou falso (false).

```
// OPERADORES LÓGICOS
// E &&
true && true; // true, ambos os valores são verdadeiros
true && false && true; // false, um dos valores é falso

// OU ||
true || true; // true, um dos valores é verdadeiro
false || false; // false, um dos valores é falso
true || false || true; // true, um dos valores é verdadeiro

// NOT !
!true; // false, nega o valor verdadeiro
!false; // true, nega o valor falso
```

# JS: operadores lógicos

| Operador | Exemplo   |
|----------|-----------|
| &&       | E (AND)   |
|          | OU (OR)   |
| !        | NÃO (NOT) |

Fonte: autora, 2026

# JS: Operador E (&&)

- ❖ Todas as **condições** testadas precisam ser verdadeiras para o resultado final ser verdadeiro.

| Condição 1 | Condição 2 | Resultado |
|------------|------------|-----------|
| true       | true       | true      |
| true       | false      | false     |
| false      | true       | false     |
| false      | false      | false     |

# JS: Operador OU (||)

- ❖ Se alguma das **condições testadas** for verdadeira, o resultado final será verdadeiro.

| Condição 1 | Condição 2 | Resultado |
|------------|------------|-----------|
| true       | true       | true      |
| true       | false      | true      |
| false      | true       | true      |
| false      | false      | false     |

# JS: Operador NÃO (!)

O operador NÃO (NOT) é utilizado para inverter um valor lógico.

- ❖ Se o valor for **true**, ele se torna false.
- ❖ Se o valor for **false**, ele se torna true.

| Valor original | Valor alterado | Resultado |
|----------------|----------------|-----------|
| true           | NÃO verdadeiro | false     |
| false          | NÃO falso      | true      |

# Expressões com operadores lógicos

```
//Expressões
// E &&
5 > 1 && 4 < 5; // true, 5 é maior que 1 E 4 é menor que 5

5 >= 5 && 4 < 5; // true, 5 é maior ou igual a 5 E 4 é menor que 5

// OU ||
5 === "5" || Number.isNaN(0/0); // true
// 5 é igual estritamente a "5" (falso) OU 0/0 é NaN (verdadeiro)

// NOT !
!4 > 3
// 4 NÃO É menor (maior) que 3?

// É a mesma coisa que escrever isso: 4 é maior que 3?
4 < 3;

// Combinações: OU || NOT !
5 === "5" || !Number.isNaN(0/0); // false
// 5 é igual estritamente a "5"? (falso, tem tipos diferentes)
// OU
// 0/0 NÃO É NaN? (falso, é NaN)
// falso + falso = falso
```



# Operadores de incremento / decremento

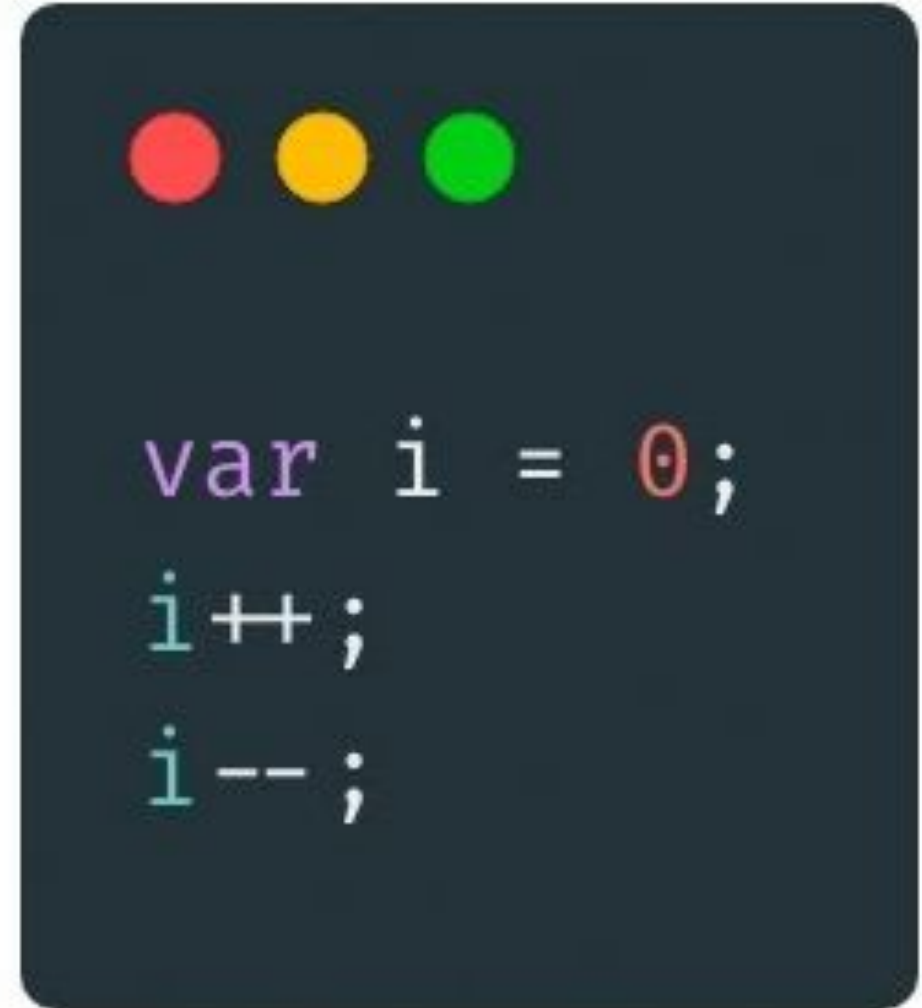
# JS: operadores de incremento e decremento

- ❖ São usados para **aumentar** ou **diminuir** valores de forma prática.

| Nome                              | Operador | Equivalência                                |
|-----------------------------------|----------|---------------------------------------------|
| Incremento (pré ou pós)           | ++       | contador = contador + 1<br>(pré-incremento) |
| Decremento (pré ou pós)           | --       | contador = contador - 1                     |
| Atribuição composta com adição    | += 6     | contador = contador + 6                     |
| Atribuição composta com subtração | -= 4     | contador = contador - 4                     |

# JS: operadores de incremento e decremento

- ❖ Os operadores de **incremento** (++) e **decremento** (--) servem para aumentar ou diminuir o valor atual de uma variável em 1.



```
var i = 0;  
i++;  
i--;
```

# JS: operador de pós-incremento

- ❖ Quando o operador aparece depois da variável (*number++*), o JavaScript primeiro utiliza o valor atual da variável e só **depois** faz o incremento (pós-incremento).

```
let number = 1;
console.log("number atualizado: " + number++);
console.log("number atualizado: " + number--);

number atualizado: 1
number atualizado: 2
```

# JS: operador de pré-incremento

- ❖ Já onde o operador aparece antes da **variável** (++number), o JavaScript primeiro realiza o incremento e **depois utiliza** o novo valor da variável (**pré-incremento**).

```
let number = 1;
console.log("number atualizado: " + ++number);
console.log("number atualizado: " + --number);

number atualizado: 2
number atualizado: 1
```

# JS: operador de pós-incremento x pré-incremento

- ❖ Isso significa que, dependendo da posição do operador, uma expressão pode retornar o valor antigo ou o valor atualizado da variável.

```
> //Comparativo
{
  let numberComparison = 1;
  console.log("valor atualizado: " + numberComparison); // 1

  // retorna o valor atual (1) e depois adiciona + 1 (2)
  console.log("valor atualizado: " + numberComparison++);

  // adiciona + 1 ao valor atual (2) e o retorna
  console.log("valor atualizado: " + ++numberComparison);
}

valor atualizado: 1
valor atualizado: 1
valor atualizado: 3
```

Depois incrementa

Primeiro mostra o valor atual

Primeiro incrementa

Depois mostra o valor atual

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

# Dúvidas?



# Vídeos complementares

- Assista os vídeos abaixo para revisar o conteúdo dessa aula.





# Exercício

## Exercício

- ❖ Agora teste você em seu navegador os scripts vistos nessa aula.
- ❖ Acesse o **arquivo de exemplo** no link abaixo.
- ❖ Copie cada bloco de código de exemplo, uma a uma na aba console das **ferramentas de desenvolvedor** do seu navegador, e analise os resultados.
- ❖ Exemplos

# Referência da aula

- ❖ FLANAGAN, David. JavaScript: O Guia Definitivo. 7. ed. Porto Alegre: Bookman, 2025. E-book. p.i. ISBN 9788582607008. Disponível em:  
<https://integrada.minhabiblioteca.com.br/reader/books/9788582607008/>. Acesso em: 29 mai. 2026.
- ❖ MOREIRA, Sabrina B. Iniciando com JavaScript: um guia prático para entender os fundamentos da linguagem da web. 1. ed. [S.l.]: Kindle Direct Publishing, 2024. E-book Kindle. Disponível em:  
<https://www.amazon.com.br/dp/B0CW9MLKTT>. Acesso em: 29 mai. 2026.

# Referência da aula

- ❖ DANTHELION. Javascript - Variáveis. Condado Braveheart GameDev, 22 jan. 2017. Disponível em: <https://condadobraveheart.com/threads/javascript-variaveis.1657/>. Acesso em: 30 mar. 2026.
- ❖ GERSCHAU, Felix. JavaScript's Memory Management Explained. Felix Gerschau, 27 out. 2020. Disponível em: <https://felixgerschau.com/javascript-memory-management/>. Acesso em: 30 mar. 2026.
- ❖ TASKER, Peter. How Unicode Works: What Every Developer Needs to Know About Strings. Delicious Brains, 12 jan. 2021. Disponível em: <https://deliciousbrains.com/how-unicode-works/>. Acesso em: 30 mar. 2026.



[univille.br](http://univille.br)